Responsive Design Deep Dive & Advanced CSS

1- Responsive images (srcset, picture)

2- Fluid typography (clamp!)

3- Container queries (cutting edge!)

4- Responsive navigation patterns

Learn it, use it, immediately!



RESPONSIVE IMAGES



The Image Problem

The Image Problem - Wasting Data & Performance

What's Actually Happening:

- Image: 3000px wide × 2000px tall
- File size: 2MB (massive!)
- Displays at: 400px on mobile (scaled down by browser)
- Result: Wasting 80% of downloaded data
- User experience: Slow loading, wasted bandwidth

The Math:

- Mobile user downloads: 2MB
- Mobile user actually sees: ~400px wide (needs ~200KB)
- Wasted data: 1.8MB (90% waste!)
- Multiply by 10 images = 18MB wasted



We can fix this with responsive image techniques

Solution 1: CSS Max-Width

The Basic Rule:

```css
img {
  max-width: 100%;
  height: auto;
}
```

How It Works:

- max-width: 100% → Image never wider than its container
- height: auto → Maintains aspect ratio (no distortion!)
- Browser scales image down (never up beyond original size)
- Works for ALL images automatically

What This Solves:

- Images never overflow containers
- No horizontal scrolling on mobile
- Maintains proper aspect ratio
- Images scale to fit any screen size
- Simple, universal solution

What This DOESN'T Solve:

- File size problem (still downloading full-size image)
- Performance issue (large file on slow connection)
- Wasted bandwidth (mobile users download desktop images)

Solution 2: srcset - Multiple Image Sizes

srcset - Let the Browser Choose the Best Image!

```
<img
  src="image-800.jpg"
  srcset="image-400.jpg 400w,
          image-800.jpg 800w,
          image-1200.jpg 1200w"
  sizes="(max-width: 600px) 400px,
         (max-width: 1000px) 800px,
         1200px"
  alt="Responsive image">
```

The Result:

Mobile (375px): Downloads image-400.jpg (50KB)
Tablet (768px): Downloads image-800.jpg (200KB)
Desktop (1920px): Downloads image-1200.jpg (500KB)

What This Does:

You provide: Multiple versions of the same image
Browser decides: Which version to download based on:

- Screen size
- Device pixel ratio (retina displays)
- Network speed (sometimes)
- User preferences

Performance Win:

- Mobile: 50KB instead of 2MB = 40× faster!
- Users download only what they need
- Faster page loads
- Less data usage
- Better user experience

Solution 3: The picture Element - Art Direction

The picture Element - Different Images, Not Just Different Sizes

```
<picture>
  <!-- Desktop: Wide landscape showing full scene -->
  <source media="(min-width: 1024px)"
          srcset="hero-landscape-wide.jpg">

  <!-- Tablet: Medium crop, balanced composition -->
  <source media="(min-width: 768px)"
          srcset="hero-landscape-medium.jpg">

  <!-- Mobile: Tight portrait crop on subject -->
  <img src="hero-portrait-tight.jpg"
       alt="Mountain landscape at sunset">
</picture>
```

When to Use picture:

- You want DIFFERENT crops/compositions for different screens
- Art direction control (what users see matters!)
- Serving modern image formats (WebP, AVIF) with fallbacks
- Different aspect ratios per device

How It Works:

1. Browser checks each `<source>` from top to bottom
2. First matching media query wins
3. Downloads that image
4. `<img>` is fallback (always required!)

picture vs srcset - When to Use Which?

Use srcset When:

- Same photo, just different file sizes
- Performance optimization (smaller files for mobile)
- Same composition/crop at all sizes
- Let browser decide based on screen + pixel density
- Most common use case (90% of situations!)

The Quick Decision:

Ask yourself:
"Am I serving the SAME image at different sizes, or DIFFERENT images?"
Same image → Use **srcset**
Different images → Use **picture**

Use picture When:

- Different crops/compositions for different screens
- Art direction control (portrait vs landscape)
- Different aspect ratios per device
- Serving modern formats (WebP) with fallbacks
- Text visibility requires different crops
- You need precise control over what users see

Solution 4: object-fit for Image Control

object-fit - Control How Images Fill Their Containers

object-fit = Consistent image sizes without distortion!

```css
img {
  width: 100%;
  height: 300px;
  object-fit: contain; /* Fits entire image inside container */
  object-fit: fill;  /* Stretches image to fill container - Avoid this! */
  object-fit: cover; /* Fills container, may crop image */
  object-fit: scale-down; /* Smaller of contain or original size */
  object-fit: none; /* Original size - may overflow container */
  object-position: top center;  /* Show top of image */
  object-position: center center;  /* Show center of image */
  object-position: bottom center;  /* Show bottom of image */
  object-position: left center;  /* Show left of image */
  object-position: right center;  /* Show right of image */
  object-position: 50% 50%;  /* Show center of image */
}
```

Solution 4: aspect-ratio - Modern CSS Magic!

Why aspect-ratio is Amazing:

- No complex math (no more padding-bottom percentages!) Clean, readable code
- Works with any unit (%, vw, px)
- Perfect for responsive images/videos
- Maintains consistency across components
- Way easier than old hacks!

How aspect-ratio Works:

- Maintains width-to-height ratio automatically
- Width is flexible, height calculated automatically
- Or height is set, width calculated automatically
- No padding-bottom hacks needed!

```css
/* Video/Widescreen */
.video {
  aspect-ratio: 16 / 9;  /* YouTube, modern video */
}

/* Square */
.square {
  aspect-ratio: 1 / 1;  /* Instagram posts, avatars */
}

/* Classic Photo */
.photo {
  aspect-ratio: 4 / 3;  /* Traditional photography */
}

/* Portrait */
.portrait {
  aspect-ratio: 3 / 4;  /* Vertical orientation */
}

/* Ultra-wide */
.ultrawide {
  aspect-ratio: 21 / 9;  /* Cinematic */
}

/* Golden Ratio */
.golden {
  aspect-ratio: 1.618 / 1;  /* Can use decimals! */
}
```

RESPONSIVE TYPOGRAPHY



Typography Problem

The Typography Problem - Fixed Sizes Don't Scale

The Core Problem:

Fixed pixel sizes don't adapt to:

- Different screen sizes (phone vs desktop)
- User preferences (accessibility)
- Viewing distances (phone up close vs monitor far away)

What We Need:

- Headings that grow with screen size
- Body text that's readable on all devices
- Smooth scaling, not sudden jumps
- Respects user accessibility preferences
- Maintains visual hierarchy



We need typography that adapts smoothly!

Solution 1: Media Query Steps

Traditional Approach - Media Query Steps

What Happens:

- Text size stays same...stays same...JUMP at 768px
- Text stays same again...stays same...JUMP at 1024px
- Creates sudden "steps" rather than smooth scaling
- Between breakpoints, no change at all

The Problem:

- Sudden jumps at specific breakpoints (not smooth)
- Text same size between breakpoints (768-1023px all identical)
- More media queries = more code to maintain
- Requires choosing "magic numbers" for breakpoints
- Not truly fluid - just three fixed sizes

This approach works, but it's not elegant!

```css
/* Mobile - Base styles */
h1 {
  font-size: 2rem;  /* 32px on mobile */
}

p {
  font-size: 1rem;  /* 16px */
}

/* Tablet - First jump */
@media (min-width: 768px) {
  h1 {
    font-size: 2.5rem;  /* 40px - sudden jump! */
  }

  p {
    font-size: 1.0625rem;  /* 17px */
  }
}

/* Desktop - Second jump */
@media (min-width: 1024px) {
  h1 {
    font-size: 3rem;  /* 48px - another jump! */
  }

  p {
    font-size: 1.125rem;  /* 18px */
  }
}
```

Solution 2:
clamp() - The Modern Way

How It Works:

- **Minimum**: 2rem (32px) - never smaller
- **Preferred**: 5vw (5% of viewport width) - scales smoothly!
- **Maximum**: 4rem (64px) - never larger
- Browser calculates automatically

Why clamp() is AMAZING:

- Smooth scaling (no jumps!)
- No media queries needed
- One line instead of many
- Truly fluid typography
- Respects user preferences (REM)
- Controls min/max for safety
- Works perfectly on any screen size

```
h1 {
  font-size: clamp(2rem, 5vw, 4rem);
  /*              min   prefer max     */
}

p {
  font-size: clamp(1rem, 2vw, 1.25rem);
}
```

How clamp() Works –

Understanding the Three Values 2

1. MINIMUM (Safety Floor)

- The smallest the font can ever be
- Used when PREFERRED is too small
- Ensures readability on tiny screens
- Usually in REM for accessibility
- Think: "What's the minimum readable size?"

2. PREFERRED (Scalable Middle)

- What you ideally want the font size to be
- Usually uses viewport units (vw, vh)
- Scales smoothly as viewport changes
- Browser uses this when it's between min and max
- Think: "How should this scale with screen size?"

3. MAXIMUM (Safety Ceiling)

- The largest the font can ever be
- Used when PREFERRED is too large
- Prevents text from becoming ridiculously huge
- Usually in REM for consistency
- Think: "What's the maximum sensible size?"

Practical clamp() Examples

```css
/* Heading levels */
h1 {
  font-size: clamp(2rem, 5vw, 4rem);
  /* 32px → scales → 64px */
}

h2 {
  font-size: clamp(1.5rem, 3.5vw, 3rem);
  /* 24px → scales → 48px */
}

h3 {
  font-size: clamp(1.25rem, 2.5vw, 2rem);
  /* 20px → scales → 32px */
}

h4 {
  font-size: clamp(1.125rem, 2vw, 1.5rem);
  /* 18px → scales → 24px */
}
```

```css
h5 {
  font-size: clamp(1rem, 1.5vw, 1.25rem);
  /* 16px → scales → 20px */
}

h6 {
  font-size: clamp(0.875rem, 1.25vw, 1rem);
  /* 14px → scales → 16px */
}

/* Body text */
p, li {
  font-size: clamp(1rem, 1.5vw, 1.125rem);
  /* 16px → scales → 18px */
}
```

The Beauty: - Set once, works everywhere! - Maintains hierarchy at all sizes - No media queries needed - Truly fluid scaling - Easy to maintain - Consistent visual rhythm

CSS Variables + clamp()

Design System Approach

Professional Design System with CSS Variables + clamp()

Benefits:

1. **Consistency**: Everyone uses same scale
2. **Maintainability**: Change once, updates everywhere
3. **Readability**: Semantic names (--font-xl vs cryptic clamp)
4. **Scalability**: Easy to add new sizes
5. **Collaboration**: Team understands the system
6. **Flexibility**: Override for specific needs

```css
:root {
  /* === TYPOGRAPHY SCALE === */
  --font-xs: clamp(0.75rem, 1vw, 0.875rem);
  --font-sm: clamp(0.875rem, 1.25vw, 1rem);
  --font-base: clamp(1rem, 1.5vw, 1.125rem);
  --font-lg: clamp(1.125rem, 2vw, 1.5rem);
  --font-xl: clamp(1.25rem, 2.5vw, 2rem);
  --font-2xl: clamp(1.5rem, 3vw, 2.5rem);
  --font-3xl: clamp(2rem, 4vw, 3rem);
  --font-4xl: clamp(2.5rem, 5vw, 4rem);
  --font-5xl: clamp(3rem, 6vw, 5rem);

  /* === SPACING SCALE === */
  --space-xs: clamp(0.25rem, 0.5vw, 0.5rem);
  --space-sm: clamp(0.5rem, 1vw, 0.75rem);
  --space-md: clamp(1rem, 2vw, 1.5rem);
  --space-lg: clamp(1.5rem, 3vw, 2.5rem);
  --space-xl: clamp(2rem, 4vw, 4rem);
  --space-2xl: clamp(3rem, 6vw, 6rem);

  /* === COLORS === */
  --color-primary: #3498db;
  --color-text: #2c3e50;
  --color-background: #ffffff;

  /* === OTHER === */
  --leading-tight: 1.2;
  --leading-normal: 1.5;
  --leading-relaxed: 1.75;
  --max-width: 1200px;
}
```

CSS Variables + clamp()

Design System Approach

Professional Design System with CSS Variables + clamp()

Benefits:

1. **Consistency**: Everyone uses same scale
2. **Maintainability**: Change once, updates everywhere
3. **Readability**: Semantic names (--font-xl vs cryptic clamp)
4. **Scalability**: Easy to add new sizes
5. **Collaboration**: Team understands the system
6. **Flexibility**: Override for specific needs

```css
/* Apply system */
body {
  font-size: var(--font-base);
  line-height: var(--leading-relaxed);
  color: var(--color-text);
}

.container {
  max-width: var(--max-width);
  padding: var(--space-lg);
}

h1 {
  font-size: var(--font-4xl);
  margin-bottom: var(--space-lg);
  line-height: var(--leading-tight);
}

.button {
  font-size: var(--font-sm);
  padding: var(--space-sm) var(--space-md);
}
```

Readable Line Lengths

The "ch" Unit - Controlling Line Length for Readability

What is "ch"?

- ch = width of the "0" (zero) character in the current font
- 1ch ≈ average width of one character
- 65ch ≈ approximately 65 characters wide
- Scales with font-size automatically!

The Magic:

- Text never too wide (prevents eye strain)
- Scales with font-size automatically
- Works on any screen size
- Centers content beautifully
- Optimal reading experience
- Respects accessibility preferences

```css
.article-container {
  max-width: 65ch; /* Shorter for body text */
  margin: 0 auto;
  padding: clamp(2rem, 5vw, 4rem);
}

.article-title {
  font-size: clamp(2rem, 5vw, 3.5rem);
  line-height: 1.2;
  margin-bottom: 1rem;
  max-width: 25ch;  /* Shorter for headlines */
}

.article-subtitle {
  font-size: clamp(1.125rem, 2vw, 1.5rem);
  line-height: 1.4;
  color: #666;
  margin-bottom: 2rem;
  max-width: 45ch;
}

.article-body {
  font-size: clamp(1rem, 1.5vw, 1.125rem);
  line-height: 1.7;
  max-width: 65ch;  /* Main content */
}

.article-body blockquote {
  max-width: 55ch;  /* Slightly narrower */
  margin: 2rem auto;
  font-size: 1.25em;
  font-style: italic;
  padding-left: 2rem;
  border-left: 4px solid #3498db;
}
```

CONTAINER QUIERIES



The Media Query Limitation

The Media Query Problem - Viewport vs Container

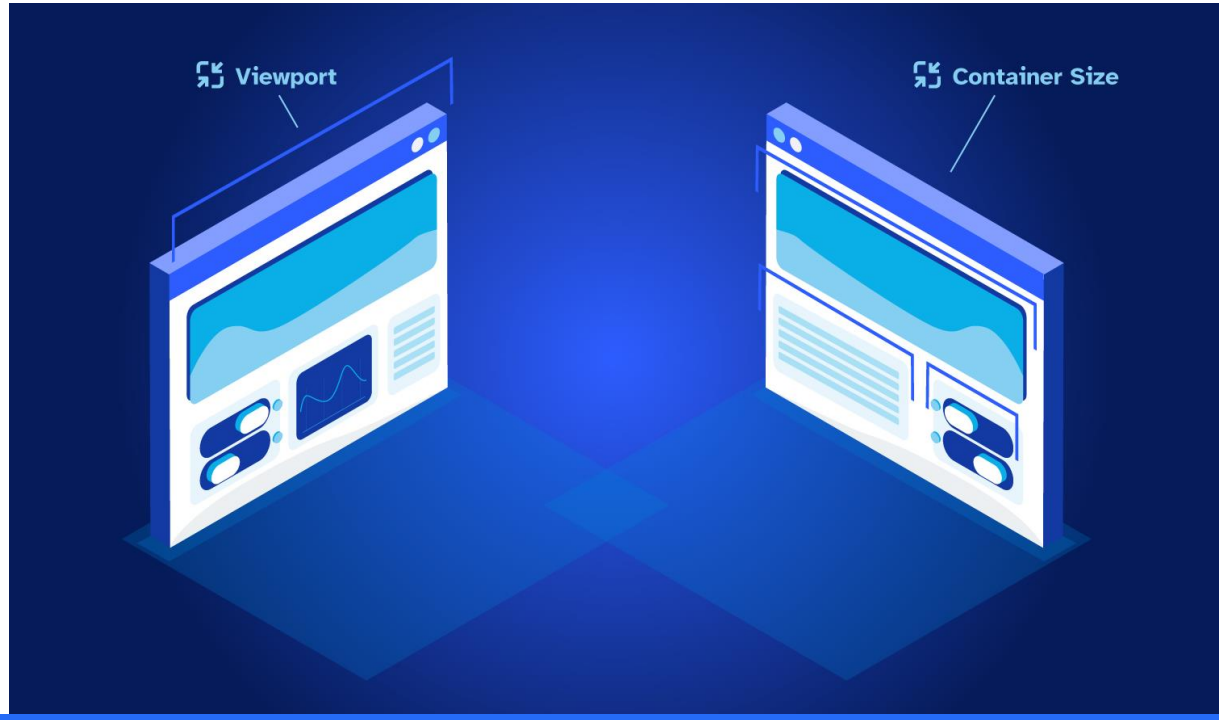
Media queries only know about:

- Viewport width (browser window)
- Viewport height
- Device orientation
- Screen resolution

Media queries DON'T know about:

- Container width (where component actually lives)
- Parent element size
- Available space for component

The Core Problem: Components need to adapt to their CONTAINER, not the viewport!



This is the limitation of media queries!

Container Queries - The Solution!

What are Container Queries?

- Query the SIZE of a container, not the viewport
- Components adapt to their available space
- Same component works perfectly in ANY context
- Truly reusable, modular components

Revolutionary Benefits:

- **True Component Reusability** • Write component once • Works in ANY container size • No context-specific classes needed
- **Context-Aware Design** • Component knows its available space • Adapts automatically • No manual class assignment
- **Simplified Code** • One component, not multiple variants • Less CSS to maintain • Cleaner HTML
- **Better Developer Experience** • Drop component anywhere • It just works • Predictable behavior
- **Component Libraries** • Perfect for design systems • Truly portable components • Framework-agnostic

```css
/* Component adapts to ITS container, not viewport! */
.card-container {
  container-type: inline-size;  /* Make this a container */
}

@container (min-width: 400px) {
  .card {
    /* Horizontal layout when container ≥ 400px */
    display: grid;
    grid-template-columns: 1fr 2fr;
  }
}

@container (max-width: 399px) {
  .card {
    /* Vertical layout when container < 400px */
    display: flex;
    flex-direction: column;
  }
}
```

Container Query Syntax - How to Use Them

The Two-Step Process:

```css
/* Step 1: Define a Container */
.card-container {
  container-type: inline-size;  /* This is a query container! */
  container-name: card-wrapper; /* Optional: give it a name */
}

/* Step 2: Query the Container */
@container card-wrapper (min-width: 400px) {
  .card {
    display: grid;
    grid-template-columns: 1fr 2fr;
  }
}
```

- @container ← Keyword (like @media)
- card-wrapper ← Container name (optional)
- (min-width: 400px) ← Condition to test
- {...} ← Styles when condition true

container-type Options:

- **inline-size** (Most Common!)
  - Queries width only (horizontal size)
  - Best for most layouts
  - Performance-friendly
- **size**
  - Queries both width AND height
  - Use when component adapts to both dimensions
  - Slightly more expensive
- **normal**
  - Not a query container (default)

Container Units (Bonus!)

Query containers get special units:

```css
.card-container {
  container-type: inline-size;
}

@container (min-width: 400px) {
  .card h3 {
    font-size: 5cqw;  /* 5% of container width! */
  }
}
```

Container query units:

- **cqw** - 1% of container width
- **cqh** - 1% of container height
- **cqi** - 1% of container inline size
- **cqb** - 1% of container block size
- **cqmin** - smaller of cqi or cqb
- **cqmax** - larger of cqi or cqb

Container Queries vs Media Queries - When to Use Each

Decision Framework: Ask yourself:

"Am I changing the overall PAGE structure?" → Use **Media Queries**

- Adding/removing sidebars
- Changing grid columns
- Overall layout shifts

"Am I adapting a COMPONENT to its space?" → Use **Container Queries**

- Card layouts
- Widgets
- Reusable components
- Items that appear in different contexts

"Do I need BOTH?" → Use Both!

- Media queries for page structure
- Container queries for components

RESPONSIVE NAVIGATION



Navigation Challenge

Navigation is one of the trickiest responsive challenges!

The Core Challenge:

Desktop navigation patterns DON'T work on mobile:

- 7+ horizontal links = 1000px+ needed
- Mobile screens = 320-428px wide
- Can't fit!

What Happens Without Responsive Navigation:

1. **Overflow** - Links run off screen, hidden
2. **Wrapping** - Multi-row navigation (messy!)
3. **Tiny targets** - Links too small to tap
4. **Horizontal scrolling** - Terrible UX

What We Need:

Different navigation patterns for different screen sizes:

Mobile (< 768px):

- Vertical stacking OR
- Hamburger menu (collapsible) OR
- Tab bar at bottom

Desktop (768px+):

- Horizontal navigation
- All links visible
- Large click targets

Pattern 1: Simple Stack

Pattern 1: Simple Stack - Easiest Solution

The Concept:

- Mobile: Stack links vertically (column)
- Desktop: Display links horizontally (row)
- Uses Flexbox + one media query
- No JavaScript needed!

When to Use:

- ✅ Small sites with 3-5 links ✅ Simple projects where hamburger is overkill ✅ When you want all links visible always ✅ Learning projects - start here before hamburger ✅ Accessible-first - no hidden content

When NOT to Use:

- ❌ Sites with 7+ navigation links ❌ When you need to conserve vertical space ❌ Complex mega-menus ❌ When following modern design conventions

Pros:

- Simplest solution - beginner-friendly
- No JavaScript required
- All links always visible
- Easy to implement and maintain
- Accessible by default
- Works with any number of links

Cons:

- Mobile nav takes vertical space
- Pushes content down on mobile
- Not ideal for many links (6+)
- Less common pattern (users expect hamburger)

Pattern 1: Simple Stack

Pattern 1: Simple Stack - Easiest Solution

The Concept:

- Mobile: Stack links vertically (column)
- Desktop: Display links horizontally (row)
- Uses Flexbox + one media query
- No JavaScript needed!

```html
<nav class="nav">
    <a href="#home" class="nav-link">Home</a>
    <a href="#about" class="nav-link">About</a>
    <a href="#services" class="nav-link">Services</a>
    <a href="#portfolio" class="nav-link">Portfolio</a>
    <a href="#contact" class="nav-link">Contact</a>
</nav>
```

```css
/* Mobile First - Vertical Stack (default) */
.nav {
    display: flex;
    flex-direction: column;
    /* Stack vertically */
    gap: 1rem;
    padding: 1rem;
    background-color: #2c3e50;
}

.nav-link {
    color: white;
    text-decoration: none;
    padding: 1rem;
    background-color: #34495e;
    border-radius: 6px;
    text-align: center;
    transition: background-color 0.2s;
}

.nav-link:hover {
    background-color: #3498db;
}

/* Desktop - Horizontal Row */
@media (min-width: 768px) {
    .nav {
        flex-direction: row;
        /* Switch to horizontal! */
        justify-content: center;
        gap: 1.5rem;
        padding: 1.5rem;
    }

    .nav-link {
        padding: 0.75rem 1.5rem;
    }
}
```

Pattern 2: Hamburger Menu - The Standard

Pattern 1: Simple Stack - Easiest Solution

The Concept:

- Mobile: Stack links vertically (column)
- Desktop: Display links horizontally (row)
- Uses Flexbox + one media query
- No JavaScript needed!

When to Use:

- ✅ Small sites with 3-5 links ✅ Simple projects where hamburger is overkill ✅ When you want all links visible always ✅ Learning projects - start here before hamburger ✅ Accessible-first - no hidden content

When NOT to Use:

- ❌ Sites with 7+ navigation links ❌ When you need to conserve vertical space ❌ Complex mega-menus ❌ When following modern design conventions

Pros:

- Simplest solution - beginner-friendly
- No JavaScript required
- All links always visible
- Easy to implement and maintain
- Accessible by default
- Works with any number of links

Cons:

- Mobile nav takes vertical space
- Pushes content down on mobile
- Not ideal for many links (6+)
- Less common pattern (users expect hamburger)

Hamburger Menu - HTML Structure

Accessibility features:

- `aria-label` on nav (describes navigation)
- `aria-label` on checkbox (screen readers)
- `aria-hidden="true"` on spans (decorative only)
- `role="list"` ensures list semantics preserved

```html
<nav class="navbar" aria-label="Main navigation">
    <!-- Logo/Brand -->
    <div class="logo">
        <a href="#home">MyBrand</a>
    </div>

    <!-- Hidden Checkbox (Controls State) -->
    <input type="checkbox" id="menu-toggle"
        class="menu-toggle" aria-label="Toggle navigation menu">

    <!-- Hamburger Icon (Label for Checkbox) -->
    <label for="menu-toggle" class="hamburger" aria-label="Menu button">
        <span aria-hidden="true"></span>
        <span aria-hidden="true"></span>
        <span aria-hidden="true"></span>
    </label>

    <!-- Navigation Menu -->
    <ul class="nav-menu">
        <li><a href="#home">Home</a></li>
        <li><a href="#about">About</a></li>
        <li><a href="#services">Services</a></li>
        <li><a href="#portfolio">Portfolio</a></li>
        <li><a href="#contact">Contact</a></li>
    </ul>
</nav>
```

1. The Container:

```
<nav class="navbar">
```

- Semantic nav element
- Class for stying
- Wrapper for everything

2. The Logo:

```
<div class="logo">
    <a href="#home">MyBrand</a>
</div>
```

- Brand name or image
- Always visible
- Links to home

3. The Hidden Checkbox (THE KEY!):

```
<input type="checkbox" id="menu-toggle" class="menu-toggle">
```

- type="checkbox" - Binary state (checked/unchecked)
- id="menu-toggle" - Connects to label
- Hidden with CSS
- Controls menu visibility

4. The Hamburger Button (Label):

```
<label for="menu-toggle" class="hamburger">
   <span></span>
   <span></span>
   <span></span>
 </label>
```

- for="menu-toggle" - Clicking label toggles checkbox!
- Three elements = three hamburger lines
- Visible on mobile, hidden on desktop
- Transforms to X when menu open

5. The Navigation Menu:

```
<ul class="nav-menu">
    <li><a href="#home">Home</a></li>
    <li><a href="#about">About</a></li>
</ul>
```

- Standard unordered list
- Hidden by default on mobile
- Shown when checkbox is checked
- Always visible on desktop

Hamburger Menu - CSS Part 1

```css
/* ===== NAVBAR CONTAINER ===== */
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 1rem 2rem;
    background-color: #2c3e50;
    position: relative;
}

/* ===== LOGO ===== */
.logo a {
    color: white;
    text-decoration: none;
    font-size: 1.5rem;
    font-weight: bold;
}

/* ===== HIDE THE CHECKBOX ===== */
.menu-toggle {
    display: none;
    /* Completely hidden! */
}

/* We'll use :checked to control menu visibility */

/* ===== HAMBURGER ICON ===== */
.hamburger {
    display: flex;
    flex-direction: column;
    gap: 5px;
    cursor: pointer;
    padding: 0.5rem;
    z-index: 1000;
    /* Above menu */
}
```

```css
/* The three bars */
.hamburger span {
    width: 25px;
    height: 3px;
    background-color: white;
    border-radius: 3px;
    transition: all 0.3s ease;
    /* Smooth animation */
}

/* ===== NAVIGATION MENU (Mobile - Hidden by default) ===== */
.nav-menu {
    position: absolute;
    top: 100%;
    /* Below navbar */
    left: 0;
    right: 0;
    background-color: #34495e;

    display: none;
    /* HIDDEN by default! */
    flex-direction: column;

    list-style: none;
    margin: 0;
    padding: 0;

    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}
```

```css
.nav-menu li {
    border-bottom: 1px solid rgba(255, 255, 255, 0.1);
}

.nav-menu a {
    display: block;
    color: white;
    text-decoration: none;
    padding: 1rem 2rem;
    transition: background-color 0.2s;
}

.nav-menu a:hover {
    background-color: #3498db;
}
```

1. Navbar Container:

```
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 1rem 2rem;
    background-color: #2c3e50;
    position: relative;
}
```

- Flexbox for layout
- Logo and hamburger at opposite ends
- Relative positioning for absolute menu

2. Hide the Checkbox:

```
.menu-toggle { display: none; }
```

- Completely hidden from view
- Still functional (can be toggled by label)
- State accessible via :checked

3. Navigation Menu (Hidden by Default):

```
.nav-menu {
    position: absolute;
    top: 100%;
    left: 0;
    right: 0;
    background-color: #34495e;
    display: none;
    flex-direction: column;
    list-style: none;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
```

- Absolute positioned below navbar
- Hidden with `display: none`
- Will show when checkbox is checked (next slide!)

Hamburger Menu - CSS Part 2

```css
/* ===== THE MAGIC: Show menu when checkbox is checked ===== */
/* This is the key line that makes everything work! */
/* ~ is the sibling selector:
    targets .nav-menu that comes after .menu-toggle */
.menu-toggle:checked~.nav-menu {
    display: flex; /* SHOW the menu! */
}

/* ===== ANIMATE HAMBURGER TO X ===== */
/* When menu is open, transform hamburger to X */
/* Top bar: Rotate and move down */
.menu-toggle:checked~.hamburger span:nth-child(1) {
    transform: rotate(75deg) translate(7px, 7px);
}

/* Middle bar: Fade out */
.menu-toggle:checked~.hamburger span:nth-child(2) {
    opacity: 0;
    transform: translateX(20px);
    /* Slide out */
}

/* Bottom bar: Rotate and move up */
.menu-toggle:checked~.hamburger span:nth-child(3) {
    transform: rotate(-75deg) translate(7px, -7px);
}
```

```css
/* ===== DESKTOP STYLES ===== */
@media (min-width: 768px) {

    /* Hide hamburger on desktop */
    .hamburger {
        display: none;
    }

    /* Show menu always on desktop */
    .nav-menu {
        position: static;
        /* Not absolute anymore */
        display: flex !important;
        /* Always visible! */
        flex-direction: row;
        /* Horizontal! */

        background-color: transparent;
        box-shadow: none;
        gap: 0.5rem;
    }

    .nav-menu li {
        border-bottom: none;
    }

    .nav-menu a {
        padding: 0.5rem 1rem;
        border-radius: 6px;
    }

    .nav-menu a:hover {
        background-color: rgba(52, 152, 219, 0.2);
    }
}
```

```
.menu-toggle:checked ~ .nav-menu { display: flex; }
```

What this means:

- `.menu-toggle:checked` = "When checkbox is checked"
- `~` = Sibling selector ("following sibling")
- `.nav-menu` = Target the menu
- `display: flex` = Show it!

Top Bar (span 1):

```
.menu-toggle:checked ~ .hamburger span:nth-child(1) {
    transform: rotate(45deg) translate(7px, 7px);
}
```

- Rotate 45 degrees clockwise
- Move right 7px and down 7px
- Creates top part of X

Middle Bar (span 2):

```
.menu-toggle:checked ~ .hamburger span:nth-child(2) {
    opacity: 0; transform: translateX(20px);
}
```

- Fade to invisible
- Slide right for smooth disappear
- Middle bar vanishes

Bottom Bar (span 3):

```
.menu-toggle:checked ~ .hamburger span:nth-child(3) {
    transform: rotate(-45deg) translate(7px, -7px);
}
```

- Rotate 45 degrees counter-clockwise
- Move right 7px and up 7px
- Creates bottom part of X

```
@media (min-width: 768px) {
  .hamburger {
    display: none; /* Hide hamburger icon */
  }
  .nav-menu {
    display: flex !important; /* Force always visible */
    flex-direction: row; /* Horizontal layout */
    position: static; /* Remove overlay positioning */
  }
}
```

What happens on desktop:

- Hamburger disappears
- Menu always visible (display: flex !important)
- Horizontal layout (row instead of column)
- No longer absolute positioned overlay

Pattern 3: Sticky Navigation - Always Accessible

Pattern 3: Sticky Navigation - Stays Visible While Scrolling

What is Sticky Navigation?

- Navigation that "sticks" to top of viewport while scrolling
- Remains accessible no matter how far user scrolls
- One CSS property: `position: sticky`
- Works with any navigation pattern (stack, hamburger, etc.)

```css
/* Sticky navbar that works with hamburger menu */
.navbar {
  position: sticky;
  top: 0;

  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;

  background-color: white;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  z-index: 100;

  transition: box-shadow 0.3s ease;
}
```


Responsive images?
Container queries?
Fluid typography?
The first PROJECT?
Navigation patterns?
Anything unclear?
HAVE ANY
QUESTIONS?